Derivative-Informed Training of Neural Operators On-the-Fly via Sketched Tangent Consistency

Shancong (Sean) Mou

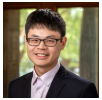
Assistant Professor
Department of Industrial and Systems Engineering
University of Minnesota, Twin Cities

SIAM NNP 2026 • Rutgers University • October 24, 2026

Joint work with



Xinhan Yang
UMN ISyE



Lu Lu
Yale SDS

Based on: *Yang, Lu, Mou. Derivative-Informed Training of Neural Operators On-the-Fly via Sketched Tangent Consistency, NeurIPS 2026.*

Outline

1. **Neural operators and why their Jacobians matter**
   - FNO / DeepONet; differentiable surrogates for PDE-CO and inverse problems
2. **Derivative-informed training of neural operators**
   - DINO, DIFNO, sensitivity-constrained FNO; offline tangent data is expensive
3. **sTCL: sketching + conditioning = on-the-fly derivative training**
   - Sketching makes it tractable; operator-aware conditioning makes it effective
   - Examples + benchmark vs. offline DI
4. **Conclusion and future direction**

Part 1 — Neural operators and why their Jacobians matter

1. **Operator learning at a glance.**
   DeepONet (Lu et al., 2021), FNO (Li et al., 2021), neural operators more broadly (Kovachki et al., 2023). They learn solution maps between function spaces.

2. **Downstream use.**
   Differentiable surrogates for PDE-constrained optimization, inverse problems, design and control — where the *Jacobian* of the surrogate matters, not just its value.

3. **Accuracy matters.**
   This talk discusses a few cost-effective explorations on improving the neural-operator accuracy.

Parametric PDEs and the solution map

Consider a parametric PDE in abstract form:

$$F(u, f) = 0, \qquad f \in \mathcal{X}, \quad u \in \mathcal{Y}.$$

When well posed, this defines a *solution operator*

$$S : \mathcal{X} \longrightarrow \mathcal{Y}, \qquad u = S(f).$$

- f – inputs (initial / boundary conditions, source terms, coefficients, geometry).
- u – the PDE solution field at one or many times.

Operator learning. Pick a neural-operator architecture S_θ and fit on $\{(f_i, u_i)\}_{i=1}^n$, where each u_i is computed offline by a classical solver:

$$\min_\theta \ \frac{1}{n} \sum_{i=1}^{n} \|S_\theta(f_i) - u_i\|_{\mathcal{Y}}^2 .$$

Once trained, S_θ is *fast*: for any new input f, inference is a single forward pass — orders of magnitude faster than a numerical solve.

Two staples: FNO and DeepONet

Fourier Neural Operator (Li et al., 2021).
Stack of integral operators in the Fourier domain:

$$v_{\ell+1}(x) = \sigma\Big(Wv_\ell(x) + \mathcal{F}^{-1}\big(R_\ell \cdot \mathcal{F}v_\ell\big)(x)\Big).$$

- Lifting $\mathcal{P} : \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_v}$ raises input channels to width d_v.
- Each Fourier layer: FFT $\to$ truncate to lowest K modes $\to$ multiply by learned $R_\ell \in \mathbb{C}^{K \times d_v \times d_v}$ $\to$ inverse FFT, plus a 1×1 residual conv W and GELU.
- Projection $\mathcal{Q} : \mathbb{R}^{d_v} \to \mathbb{R}^{d_{\text{out}}}$ reads off the prediction.

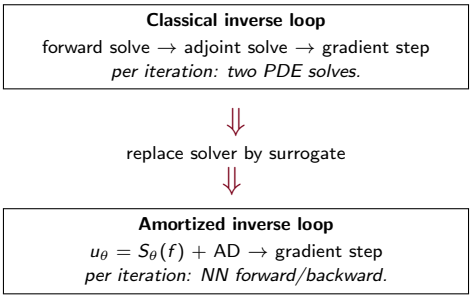
DeepONet (Lu et al., 2021).
Two-network factorization of the operator S:

$$S_\theta(f)(x) = \sum_{k=1}^{p} \underbrace{b_k\big(f|_{\{x_j\}_{j=1}^m}\big)}_{\text{branch}} \underbrace{t_k(x)}_{\text{trunk}}.$$

- Input function f sampled at *sensors* $\{x_j\}_{j=1}^m$ feeds the branch; trunk produces a position-dependent basis.
- Inspired by the universal-approximation theorem for operators (Chen & Chen, 1995).

Together, FNO and DeepONet cover most operator-learning surrogate work in the PDE community.

Once trained: fast inference, fast inverse



Cost amortizes once S_θ is trained.

Forward use. For any new input f (viscosity field, source, initial condition), evaluate $u_\theta = S_\theta(f)$ in milliseconds.

Inverse / PDE-constrained optimization. Many real problems are of the form

$$\min_{f \in U_{\mathrm{ad}}} J\big(S(f), f\big) \quad \text{s.t. PDE.}$$

Examples: recover a coefficient field from sparse measurements; design a source / control toward a target state; maximize / minimize a QoI under physical constraints.

With a differentiable surrogate, each inverse step is a gradient step on f minimising the surrogate-reduced objective $\Phi_\theta(f) = J(S_\theta(f), f)$, with the hypergradient $\nabla_f \Phi_\theta(f)$ computed by automatic differentiation through S_θ.

The reduced gradient depends on the surrogate Jacobian

For a reduced objective $\Phi(f) = J\big(S(f), f\big)$, the chain rule gives

$$\nabla_f \Phi(f) = \underbrace{DS(f)^*}_{\text{Jacobian adjoint}} \nabla_u J\big(S(f), f\big) \; + \; \nabla_f J\big(S(f), f\big).$$

Replacing S by S_θ:

$$\nabla_f \Phi_\theta(f) = \underbrace{DS_\theta(f)^*}_{\text{learned}} \nabla_u J\big(S_\theta(f), f\big) \; + \; \nabla_f J\big(S_\theta(f), f\big).$$

Key observation. For optimization, the *accuracy of the Jacobian* matters just as much as the accuracy of the values.

$\Rightarrow$ **We need to supervise the surrogate's Jacobian, not only its value.**

Part 2 — Sobolev / derivative-informed training

Sobolev training (Czarnecki et al., 2017; Lu et al., 2022): augment value supervision with derivative supervision,

$$\mathcal{L}_{\mathrm{Sob}}(\theta) = \frac{1}{n}\sum_{i=1}^{n}\Big[\,\|S_\theta(f_i) - u_i\|^2 + \mu\,\|DS_\theta(f_i) - DS(f_i)\|_{\mathrm{F}}^2\Big].$$

Why it's powerful for neural operators.

- The Fréchet derivative $DS(f)$ has a physical meaning: *sensitivity* of the solution field to a perturbation in the input function.
- For PDE-CO and inverse problems, the surrogate's Jacobian is precisely the object multiplied into the gradient.
- Empirically (DINO, DIFNO, S-FNO): training with Jacobian supervision improves both *value* and *Jacobian* accuracy of the learned operator.

DINO / DIFNO / sensitivity-constrained FNO

DINO (O'Leary-Roseberry et al., 2024) — Derivative-Informed Neural Operator on DeepONet.
DIFNO (Yao et al., 2025) — Derivative-Informed FNO.
Sensitivity-constrained FNO (Behroozi et al., 2025) — same family, different reduction basis.

Common recipe.

1. **Pre-generate Jacobian (Jacobian–vector products) offline** by solving the *forward sensitivity equation* r times per sample.
2. During training, match surrogate JVPs to the stored (reduced) tangent labels.

Generating the whole Jacobian (JVP dataset) is expensive. Smart ideas by DINO / DIFNO / SC-FNO: KLE (inputs) + POD (outputs) reduced bases, so only a small number of leading tangent directions need to be computed and stored per sample.

Research question.

Can we **further reduce the derivative-data generation burden** *and make derivative-informed training* **on the fly***?*

Part 3 — straight-forward idea: forward sensitivity equation as a physics-informed loss

Start from the parametric PDE $F(u, f) = 0$ and differentiate at $u = S(f)$ with respect to the input f:

$$\underbrace{D_u F(u, f)}_{\mathcal{A}(u,f)} \, DS(f) \; + \; \underbrace{D_f F(u, f)}_{\mathcal{B}(u,f)} \; = \; 0.$$

- $\mathcal{A}(u, f) := D_u F(u, f)$ is the *state tangent operator*.
- $\mathcal{B}(u, f) := D_f F(u, f)$ describes how the PDE residual reacts to input perturbations.
- Solving this equation gives the true solution-map Jacobian $DS(f)$.

Idea. Replace $DS(f)$ by the surrogate's Jacobian $DS_\theta(f)$ and ask the same equation to hold (residual form):

$$R_\theta(f) \; = \; \mathcal{A}\big(S_\theta(f), f\big) \, DS_\theta(f) \; + \; \mathcal{B}\big(S_\theta(f), f\big).$$

If $S_\theta = S$, then $R_\theta(f) \equiv 0$ — a data-free consistency condition for the learned Jacobian.

The most direct loss is the squared Frobenius norm of the residual matrix, evaluated over a minibatch:

$$\mathcal{L}_{\mathrm{raw}}(\theta) \; = \; \mathbb{E}_{f \sim \rho}\big[\|R_\theta(f)\|_{\mathrm{F}}^2\big].$$

Two failure modes if we try this naively

Unfortunately... $\mathcal{L}_{\mathrm{raw}}(\theta) = \mathbb{E}_f[\|R_\theta(f)\|_{\mathrm{F}}^2]$ is not compatible with neural-network training, in two aspects.

Failure mode (a): materializing the Jacobian.

$$DS_\theta(f) \in \mathbb{R}^{d_u \times d_f}.$$

Calculating a single Jacobian per sample needs $\mathcal{O}(d_f)$ forward passes. *Incompatible* with stochastic mini-batch training.

Failure mode (b): poor conditioning of the residual loss.
If $\mathcal{A}$ has stiff or indefinite spectrum, the gradient signal is poorly scaled or misleading. Hard for first-order optimization algorithms to make progress.

Fix (a): random directional sketching

Probe $R_\theta(f)$ with random directions $v \sim \nu$ in input space and look at the directional residual

$$r_\theta(f, v) \;=\; R_\theta(f)\, v \;=\; \mathcal{A}\big(S_\theta(f), f\big)\, DS_\theta(f)[v] + \mathcal{B}\big(S_\theta(f), f\big)\, v.$$

Population sTCL loss (**sketched tangent consistency loss** — enforce the forward sensitivity equation in expectation over random input-space directions):

$$\mathcal{L}_{sTCL}(\theta) \;=\; \mathbb{E}_{f\sim\rho}\, \mathbb{E}_{v\sim\nu}\Big[\, \|r_\theta(f, v)\|_{\mathcal{Z}}^2 \Big].$$

- ρ: distribution over PDE inputs f (the training distribution).
- ν: *sketch distribution* over input-space directions — typically random samples, unit-normalised.
- Each evaluation needs only one JVP $DS_\theta(f)[v]$, not the full Jacobian.

Next: how this population loss connects to a tractable mini-batch estimator.

Fix (a) cont'd: empirical sTCL estimator

Empirical sTCL estimator. For minibatch $\mathcal{D}$ (size $n_b = |\mathcal{D}|$) and q directions per sample:

$$\widehat{\mathcal{L}}_{sTCL}(\theta; \mathcal{D}) = \frac{1}{n_b\, q} \sum_{f_i \in \mathcal{D}} \sum_{k=1}^{q} \Big\| \mathcal{A}\big(S_\theta(f_i), f_i\big)\, DS_\theta(f_i)[v_{i,k}] + \mathcal{B}\big(S_\theta(f_i), f_i\big)\, v_{i,k} \Big\|_{\mathcal{Z}}^2 .$$

Total training objective (EMA-normalized weight).

$$\widehat{\mathcal{L}}_{\text{total}}(\theta; \mathcal{D}) \;=\; \widehat{\mathcal{L}}_{\text{data}}(\theta; \mathcal{D}) \;+\; \gamma_t\, \widehat{\mathcal{L}}_{sTCL}(\theta; \mathcal{D}), \qquad \gamma_t = \lambda\, \overline{\mathcal{L}}_{\text{data},t} \big/ \max\big(\overline{\mathcal{L}}_{sTCL,t}, \varepsilon\big).$$

- Each $DS_\theta(f_i)[v_{i,k}]$ is one `torch.func.jvp` call (forward-mode AD).
- $\mathcal{A}$ and $\mathcal{B}$ are *known* tangent operators.
- Per-sample cost scales with q, not with d_f:

$$\mathrm{Cost}\big(\widehat{\mathcal{L}}_{sTCL}\big) \approx q \cdot \mathrm{Cost}(\mathrm{JVP}_{S_\theta}) + q \cdot \mathrm{Cost}\big(\text{apply } \mathcal{A}, \mathcal{B}\big) + \mathrm{Cost}(\text{backward}).$$

- In our experiments, $q = 4$ is sufficient.

Why sketching gives the right expectation

Let $R_\theta(f)$ denote the linear map $v \mapsto r_\theta(f, v)$:

$$R_\theta(f)\, v \;=\; A_\theta(f)\, J_\theta(f)\, v + B_\theta(f)\, v,$$

where $J_\theta(f) = DS_\theta(f)$ is the discrete surrogate Jacobian.

Proposition (Hutchinson trace identity, applied to $R_\theta^\top R_\theta$)

If $\mathbb{E}[vv^\top] = I$, then

$$\mathbb{E}_v \, \|R_\theta(f)\, v\|_2^2 \;=\; \|R_\theta(f)\|_{\mathrm{F}}^2 .$$

Interpretation. sTCL enforces the forward sensitivity equation *in expectation over random input directions.* **Parallel with PINNs.**

- PINNs: randomly sample *spatio-temporal* points, enforce the PDE residual on average.
- sTCL: randomly sample *input-space directions*, enforce the forward sensitivity equation on average.

$\Rightarrow$ *sTCL is the derivative-level analogue of PINN collocation training.*

Comparison of raw-sTCL with offline DI

Question. Can we just drop in a raw L^2 tangent residual and be done?

Below (earlier 500-epoch runs, superseded by the final tables): 3 PDE benchmarks at $N = 512$, comparing pure FNO, *raw-L^2* sTCL (no preconditioning), and an offline derivative-informed (offline DI) reference.[1]

PDE	Pure FNO	raw-L^2 *s*TCL	Offline DI
2D Helmholtz (indefinite elliptic)	12.31%	diverges	7.91%
2D Nonlinear diffusion (SPD elliptic)	5.39%	$\sim$4.5% (no gain)	2.48%
Viscous Burgers 1D (parabolic / advective)	0.77%	0.65% (works)	0.70%

$\Rightarrow$ **Conditioning is the missing ingredient.**

[1] Our offline-DI implementation uses the *same random-Matérn directions* and the *same per-sample derivative-sampling budget* as *s*TCL, so the comparison isolates online-vs-offline supervision under matched probes (not the original KLE-input/POD-output reduced basis).

Why the raw residual loss is hard to optimize

The raw residual loss $\ell_{\mathrm{raw}}(w) = \|Aw + b\|^2$ has Hessian

$$H \;=\; 2A^\top A, \qquad \kappa(H) \;=\; \kappa(A)^2.$$

The squaring *doubles* the conditioning penalty inherited from the tangent operator A.

Cleanest way to see why this hurts: GD dynamics on a quadratic. Consider $f(x) = \frac{1}{2}x^\top Hx$ with $H \succ 0$ and $\kappa(H) = \lambda_{\max}/\lambda_{\min}$. Gradient descent $x_{k+1} = (I - \eta H)x_k$ decomposes in the eigen-directions of H:

$$x_{k+1}^{(i)} \;=\; (1 - \eta\,\lambda_i)\,x_k^{(i)}.$$

Stepsize is set by the worst (largest) mode. Stability forces

$$\eta \;=\; \mathcal{O}\big(1/\lambda_{\max}\big),$$

otherwise the stiff component blows up.

Soft modes barely move. With $\eta = \mathcal{O}(1/\lambda_{\max})$, the contraction along a low-curvature direction is

$$1 - \eta\,\lambda_{\min} \;=\; 1 - \mathcal{O}\big(\lambda_{\min}/\lambda_{\max}\big) \;=\; 1 - \mathcal{O}\big(1/\kappa(H)\big),$$

so progress along flat directions takes $\mathcal{O}(\kappa(H))$ iterations.

$\Rightarrow$ For sTCL, $\kappa(H) = \kappa(A)^2$, so the physically-important soft sensitivity modes take $\mathcal{O}(\kappa(A)^2)$ iterations — essentially frozen for first-order optimizers.

Preconditioned residual loss

We define an *operator-aware* loss:

$$\ell_M(w) \;=\; \langle Aw + b,\, M(Aw + b)\rangle_{\mathcal{Z}}\,, \qquad M \succeq 0.$$

Local Hessian: $\nabla_w^2 \ell_M = 2A^\top MA$.

Design goal. Choose a cheap M such that

$$\kappa(A^\top MA) \;\ll\; \kappa(A^\top A).$$

- Ideally $M \approx A^{-1}$ — but that would amount to solving the tangent equation.
- Instead, use a *cheap, spectrally equivalent approximation*.

For SPD elliptic tangent operators.
Fourier (or DST) shifted-Laplacian preconditioners. A single $O(N \log N)$ FFT and a diagonal scaling. No iteration.

For symmetric indefinite tangent operators (Helmholtz).
Positive M alone is not enough — $A^\top MA$ still has the $\sigma_{\min}^2$ factor along near-null modes. We need a short Krylov (MINRES) solve to build a target (next slide).

MINRES target matching for indefinite cases

Let M_0 be a cheap base preconditioner (e.g. shifted-Laplacian via DST). Let

$$P_m(A, M_0) \;\approx\; A^{-1}$$

be the action of m steps of preconditioned MINRES with preconditioner M_0.

Write the tangent equation as $Aw = g$ with $g = -b$, build an on-the-fly target, and match it (*MINRES target matching*):

$$\hat{w}_m \;=\; P_m(A, M_0)\, g \;\approx\; A^{-1} g, \qquad \ell_{\text{MINRES-target}}(w) \;=\; \|w - \hat{w}_m\|_{\mathcal{Y}}^2 .$$

- Crucially, m is *small and fixed* (we use $m = 5$ throughout).
- P_m is a **cheap on-the-fly target constructor**, not an accurate tangent solver; $\hat{w}_m$ is discarded after each update — no stored labels.

Example 1: 2D nonlinear diffusion — setup

Let $\Omega = (0,1)^2$ be the unit square with boundary $\partial\Omega$. For each input $a : \Omega \to \mathbb{R}$, the state $u : \Omega \to \mathbb{R}$ solves

$$-\nabla\cdot\left(e^{a(x)}\,\nabla u(x)\right) + u(x)^3 \;=\; f(x), \qquad x \in \Omega, \qquad u|_{\partial\Omega} = 0.$$

- *Input field* a: log-diffusivity, drawn from a fixed Matérn Gaussian-random-field prior on Ω.
- *Source* f: fixed four-bump Gaussian (same f for all training samples).
- *Operator to learn*: $S_\theta : a \mapsto u$.

Tangent equation. For input perturbation δa,

$$\underbrace{-\nabla\cdot(e^a\nabla\delta u) + 3u^2\delta u}_{\mathcal{A}(u,f)\,\delta u} \;=\; \underbrace{\nabla\cdot(e^a\delta a\,\nabla u)}_{-\mathcal{B}(u,f)\,\delta a}.$$

$\mathcal{A}$ is SPD on $H_0^1(\Omega)$.

Example 1: 2D nonlinear diffusion — preconditioner & results

Preconditioner. Cheap scaled DST shift:

$$M = D^{-1/2}\,(-\Delta_h)^{-1}\,D^{-1/2}, \qquad D = e^{a}.$$

M is SPD and approximates A^{-1} since the leading part of A is $\approx D(-\Delta)$. One DST + a diagonal multiply per JVP — no iteration.

Empirical check (at $N = 1024$).

- Pure FNO: 2.98% (function), 10.88% (Jacobian).
- Preconditioned sTCL (λ=1): **1.65%** (function), **6.07%** (Jacobian).
- Offline derivative-informed baseline: 1.07% (function), 3.80% (Jacobian).

$\Rightarrow$ A one-shot spectral preconditioner cuts both pure-FNO errors by $\sim$45% with no tangent-data pipeline; offline DI is still lower here.

Example 2: 2D Helmholtz (indefinite) — setup

Let $\Omega = (0,1)^2$ with boundary $\partial\Omega$. For each input $a : \Omega \to \mathbb{R}$, the state $u : \Omega \to \mathbb{R}$ solves

$$\left(-\Delta - \kappa^2\, e^{2a(x)}\right) u(x) \;=\; f(x), \qquad x \in \Omega, \qquad u|_{\partial\Omega} = 0.$$

- *Input field* a: log-wavenumber modulation, Matérn GRF on Ω.
- *Source* f: fixed.
- *Wavenumber*: $\kappa = 5$.
- *Operator to learn*: $S_\theta : a \mapsto u$.

Tangent equation.

$$\underbrace{\left(-\Delta - \kappa^2 e^{2a}\right)}_{\mathcal{A}(u,f)} \delta u \;=\; 2\kappa^2 e^{2a}\, \delta a\, u.$$

The state tangent operator $\mathcal{A} = -\Delta - \kappa^2 e^{2a} I$ is symmetric but *indefinite*: at $\kappa = 5$ the shift sits close to a low Dirichlet mode $\Rightarrow$ near-resonant directions with small signed eigenvalues.

Example 2: 2D Helmholtz — MINRES target & results

MINRES-target sTCL (MINRES-5 with shifted-Laplacian M_0).

- Build $\hat{w} = P_5(\mathcal{A}, M_0)\, g \approx \mathcal{A}^{-1} g$ on the fly, with $g = 2\kappa^2 e^{2a}\delta a\, u$ and $M_0 = (-\Delta_h + 0.5\kappa^2 I)^{-1}$ applied by DST-I; train $DS_\theta(a)[\delta a]$ toward $\hat{w}$.
- $m = 5$ is within seed variance of $m = 10$ and $m = 25$ on both errors (ablation in next slide).
- The MINRES target is a *cheap conditioning mechanism*, not a tangent solver; it is discarded after each update.

Empirical check (at $N = 1024$).

- Pure FNO: 2.09% (function), 12.17% (Jacobian).
- MINRES-target *s*TCL: **1.51%** (function), **9.17%** (Jacobian).
- Offline derivative-informed baseline: 1.83% (function), 10.67% (Jacobian) — *s*TCL has the lower mean on both, with no tangent-data pipeline.

Main result: function-value error vs. offline derivative-informed

Five PDEs $\times$ three data regimes $= 15$ matched-budget cells; test relative error (%), mean $\pm$ std over three seeds.
Within each PDE: same FNO backbone, Adam lr$= 10^{-3}$, $B = 32$, $q = 4$; 2000 epochs (Helmholtz, nonlinear diffusion, Burgers), 1500 (Allen–Cahn), 5000 (Navier–Stokes, velocity error).
Both *s*TCL and the offline baseline sample probe directions *identically* (matched PDE-specific probes).[2]

PDE	N	Pure FNO	*s*TCL (operator-conditioned)	Offline DI
1D Burgers	32	17.75 ± 3.10	7.65 ± 0.70	8.14 ± 0.48
	128	3.01 ± 0.24	1.72 ± 0.07	1.69 ± 0.05
	512	0.58 ± 0.06	0.52 ± 0.09	0.57 ± 0.03
1D Allen–Cahn	512	0.94 ± 0.02	0.59 ± 0.06	0.60 ± 0.04
	1024	0.50 ± 0.01	0.40 ± 0.03	0.35 ± 0.01
	2048	0.27 ± 0.01	0.25 ± 0.03	0.27 ± 0.01
2D Helmholtz	128	7.95 ± 0.54	7.48 ± 0.54	7.13 ± 0.90
	512	3.92 ± 0.46	2.25 ± 0.31	3.08 ± 1.37
	1024	2.09 ± 0.25	1.51 ± 0.17	1.83 ± 0.75
2D Nonlin. diffusion	128	9.71 ± 0.15	4.77 ± 0.17	3.43 ± 0.16
	512	4.49 ± 0.16	2.26 ± 0.07	1.52 ± 0.03
	1024	2.98 ± 0.05	1.65 ± 0.03	1.07 ± 0.02
2D Navier–Stokes	512	0.619 ± 0.009	0.400 ± 0.029	0.324 ± 0.013
	1024	0.392 ± 0.003	0.283 ± 0.016	0.236 ± 0.009
	2048	0.264 ± 0.012	0.198 ± 0.042	0.172 ± 0.005

[2] **Scope.** This is not a claim of beating every optimized DIFNO variant: we use matched PDE-specific probes on both sides, *not* the original KLE-input/POD-output reduced basis. The comparison isolates *online-vs-offline supervision under matched probes*.

Main result: Jacobian relative error

Per-(sample, direction) average $\bar{E} = \dfrac{1}{N\, n_{\mathrm{dirs}}} \displaystyle\sum_{i=1}^{N} \sum_{d=1}^{n_{\mathrm{dirs}}} \dfrac{\| J(a_i)v_d - J_\theta(a_i)v_d \|_2}{\| J(a_i)v_d \|_2}$, where $J(a_i)v_d$ is the true JVP and $J_\theta(a_i)v_d$ is the surrogate JVP (`torch.func.jvp`).

Same models as previous table; %, mean $\pm$ std over three seeds (Navier–Stokes: velocity JVP).

PDE	N	Pure FNO	sTCL (operator-conditioned)	Offline DI
1D Burgers	32	749.68 ± 80.79	74.76 ± 0.87	67.83 ± 0.80
	128	296.90 ± 18.72	73.80 ± 0.50	70.58 ± 0.22
	512	182.02 ± 16.97	$\mathbf{73.53 \pm 0.60}$	178.08 ± 15.45
1D Allen–Cahn	512	15.64 ± 1.22	3.82 ± 0.33	3.64 ± 0.32
	1024	11.59 ± 1.58	2.83 ± 0.30	2.11 ± 0.09
	2048	10.77 ± 1.47	1.69 ± 0.17	1.48 ± 0.13
2D Helmholtz	128	51.64 ± 1.48	47.78 ± 2.41	45.99 ± 4.64
	512	26.01 ± 3.04	13.83 ± 2.64	19.52 ± 9.03
	1024	12.17 ± 0.92	9.17 ± 1.51	10.67 ± 3.75
2D Nonlin. diffusion	128	29.74 ± 0.55	16.06 ± 0.62	11.65 ± 0.50
	512	15.87 ± 0.30	8.38 ± 0.15	5.36 ± 0.05
	1024	10.88 ± 0.17	6.07 ± 0.05	3.80 ± 0.12
2D Navier–Stokes	512	1.084 ± 0.023	0.570 ± 0.024	0.447 ± 0.013
	1024	0.726 ± 0.005	0.458 ± 0.024	0.278 ± 0.008
	2048	0.503 ± 0.022	0.282 ± 0.014	0.249 ± 0.007

Preconditioning ablation (2D Helmholtz N=512)

Varying only the number of online MINRES inner steps m; 2000 epochs, lr 10^{-3}, $\lambda = 0.1$ held fixed (the main N=512 cell later used the validation-selected $\lambda = 0.5$).

m	Function rel. err. (%)	Jacobian rel. err. (%)	Training wall (s)
1	3.50 ± 0.45	23.03 ± 2.52	2056
2	2.80 ± 0.20	18.32 ± 1.48	2132
3	2.52 ± 0.22	15.52 ± 1.56	2228
5	2.47 ± 0.28	14.71 ± 2.09	2412
10	2.46 ± 0.21	14.61 ± 1.65	2884
25	2.41 ± 0.15	14.51 ± 1.22	4216

- Most of the gain arrives by $m = 3$; $m = 5$ is within seed variance of $m = 10, 25$ on both errors at much lower wall-clock (2412 s vs. 4216 s for $m = 25$).
- The MINRES target is a *conditioning mechanism*, not an accurate tangent solver.
- This is the central principle: *we only need "good-enough" descent directions for the current training stage.*

Wall-clock and offline budget

Per-training-job wall-clock at $N = 512$, full training budget (2000 epochs; Allen–Cahn 1500; Navier–Stokes 5000), single RTX PRO 6000 Blackwell GPU.

Offline-DI column excludes the offline tangent-data generation stage (single-core CPU: ∼10 min for Helmholtz up to ∼13 h for Burgers and Allen–Cahn).

PDE	Pure FNO	*s*TCL (online)	Offline DI (training only)
2D Helmholtz	192 s	2832 s (MINRES-5)	2036 s
2D Nonlin. diffusion	192 s	1976 s (DST-precond.)	1796 s
Burgers (1D)	408 s	2144 s (normalized residual)	2208 s
Allen–Cahn (1D)	291 s	2295 s (raw L^2)	1911 s
Navier–Stokes (2D)	582 s	9990 s (Leray–Oseen)	4810 s

sTCL pays none of the offline tangent-data generation/storage cost

Part 3 takeaway

Sketching + conditioning =
on-the-fly derivative-informed training.

- *Sketching* (random input directions, JVP via forward AD) makes the derivative penalty stochastic-trainable.
- *Conditioning* (operator-aware preconditioners) makes the on-the-fly residual signal optimization-effective for stiff, ill-conditioned, or indefinite tangent operators.

What we claim — and what we don't.

sTCL is an online, architecture-agnostic way to get a training-quality sensitivity signal without storing tangent labels (not a universal replacement for all derivative-informed training).

- **Well-conditioned case + cheap preconditioner:** parabolic Burgers and Allen–Cahn (unpreconditioned space–time residual); SPD elliptic nonlinear diffusion (one DST + diagonal scale); Navier–Stokes saddle point (FFT Leray–Oseen weighting). 
- **Hard / indefinite case:** symmetric-indefinite Helmholtz — needs a few inner MINRES steps to build the target, but m=5 is enough. 

Sketched tangent consistency loss (sTCL):
a physics-informed way to incorporate derivative information.

1. **Not an amortization of offline data cost to online.**
   It is a *physics-informed* way of incorporating the derivative information — the forward sensitivity equation enforced in expectation over sketched input directions, with cheap operator-aware preconditioning to make first-order optimization effective.

2. **Plug-and-play.**
   *s*TCL can be used as a plug-and-play loss term in neural-operator training.

Thank you!

Shancong (Sean) Mou
Assistant Professor
Department of Industrial and Systems Engineering
University of Minnesota, Twin Cities

Email: mou00006@umn.edu
Web: https://mou-group-umn.github.io

Questions?

Acknowledgement. Thanks to my collaborators **Xinhan Yang** (Ph.D. student, UMN ISyE) and **Prof. Lu Lu** (Yale University). We acknowledge support from the **NVIDIA Academic Grant Program**. Part of this work was done during S. Mou's visit to the **Institute for Pure and Applied Mathematics (IPAM)**.

References (1/2) — operator learning & Sobolev training

1. Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar. **Fourier Neural Operator for Parametric Partial Differential Equations.** *ICLR 2021*.

2. L. Lu, P. Jin, G. Pang, Z. Zhang, G. E. Karniadakis. **Learning Nonlinear Operators via DeepONet Based on the Universal Approximation Theorem of Operators.** *Nature Machine Intelligence*, 3:218–229, 2021.

3. S. Wang, H. Wang, P. Perdikaris. **Learning the Solution Operator of Parametric PDEs with Physics-Informed DeepONets.** *Science Advances*, 7(40):eabi8605, 2021.

4. W. M. Czarnecki, S. Osindero, M. Jaderberg, G. Swirszcz, R. Pascanu. **Sobolev Training for Neural Networks.** *NeurIPS 2017*.

5. Y. Lu, J. Blanchet, L. Ying. **Sobolev Acceleration and Statistical Optimality for Learning Elliptic Equations via Gradient Descent.** *NeurIPS 2022*.

6. T. O'Leary-Roseberry, P. Chen, U. Villa, O. Ghattas. **Derivative-Informed Neural Operator (DINO): An Efficient Framework for High-Dimensional Parametric Derivative Learning.** *Journal of Computational Physics*, 2024.

7. B. Yao, D. Luo, L. Cao, N. Kovachki, T. O'Leary-Roseberry, O. Ghattas. **Derivative-Informed Fourier Neural Operator: Universal Approximation and Applications to PDE-Constrained Optimization.** arXiv:2512.14086, 2025.

8. A. Behroozi, C. Shen, D. Kifer. **Sensitivity-Constrained Fourier Neural Operators for Forward and Inverse Problems in Parametric Differential Equations.** *ICLR 2025*.

References (2/2) — PINNs, sketching, conditioning

8. M. Raissi, P. Perdikaris, G. E. Karniadakis. **Physics-Informed Neural Networks.** *Journal of Computational Physics*, 378:686–707, 2019.

9. M. F. Hutchinson. **A Stochastic Estimator of the Trace of the Influence Matrix for Laplacian Smoothing Splines.** *Comm. Stat. Simul. Comput.*, 18(3):1059–1076, 1989.

10. T. De Ryck, F. Bonnet, S. Mishra, E. de Bézenac. **An Operator Preconditioning Perspective on Training in Physics-Informed Machine Learning.** *ICLR 2024*.

11. S. Wang, X. Yu, P. Perdikaris. **When and Why PINNs Fail to Train: A Neural Tangent Kernel Perspective.** *Journal of Computational Physics*, 2022.

12. S. Wang, Y. Teng, P. Perdikaris. **Understanding and Mitigating Gradient Pathologies in Physics-Informed Neural Networks.** *SIAM Journal on Scientific Computing*, 2021.

13. A. Krishnapriyan, A. Gholami, S. Zhe, R. Kirby, M. W. Mahoney. **Characterizing Possible Failure Modes in Physics-Informed Neural Networks.** *NeurIPS 2021*.

14. N. Halko, P.-G. Martinsson, J. A. Tropp. **Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions.** *SIAM Review*, 53(2):217–288, 2011.

Backup: Fourier / DST shifted-Laplacian preconditioner

Setting. The sensitivity equation has the form

$$L\,\delta u \;=\; r,$$

where L is an SPD elliptic tangent operator (e.g. from nonlinear diffusion). Working with L^{-1} exactly would be solving the tangent equation; we want a *cheap* approximate inverse.

The approximation. Replace L^{-1} by P^{-1} where

$$P \;=\; \alpha(-\Delta_h) + \beta I.$$

This is a *shifted Laplacian* — discrete Laplacian plus a positive shift βI. Spectrally equivalent to L for the problems we care about, but *much* easier to invert.

Why it's cheap on a regular grid. $-\Delta_h$ is diagonalised by a fast spectral transform:

- *Periodic BCs*: the FFT.
- *Dirichlet BCs*: the DST (discrete sine transform).

In that basis $P = Q^\top \Lambda Q$ with Λ diagonal, so $P^{-1} r = Q^\top \Lambda^{-1} Q r$.

Backup: applying P^{-1} in three steps

Applying the shifted-Laplacian preconditioner P^{-1} to a vector r:

1. **Transform** r with FFT (or DST): $\widetilde{r} = Q\,r$.
2. **Divide** each frequency component by the shifted-Laplacian eigenvalue:

$$\widetilde{z}_j \;=\; \frac{\widetilde{r}_j}{\alpha\lambda_j + \beta}.$$

3. **Inverse transform**: $z = Q^\top \widetilde{z}$.

Then $z \approx L^{-1} r$.

Cost.

$$\mathcal{O}(N \log N) \text{ for the FFT/DST } + \; \mathcal{O}(N) \text{ for the diagonal scaling.}$$

No CG / GMRES inner loop, no iteration.

In the sTCL loss. Use P^{-1} as a left preconditioner on the residual,

$$\ell_P(\delta u) \;=\; \|L\,\delta u - r\|^2_{P^{-1}} \;=\; \big\langle L\,\delta u - r,\; P^{-1}(L\,\delta u - r)\big\rangle.$$

Hessian becomes $L^\top P^{-1} L \approx L$, restoring $\kappa(L)$ in place of $\kappa(L)^2$.